INFS3201 – Web Technologies II
Due: March 26, 2026 @ 1159pm

Assignment 4

Overview

In this assignment, you will refactor your Assignment 3 scheduling system in the following ways:

- Refactor the database design to better reflect a non-relational MongoDB data model.
- Introduce user authentication and session management, including a security access log.
- Add employee photos to the system and ensure appropriate access control.

Allowed Code

The existing restrictions of .filter(), .map(), .reduce(), etc. are still present for this assignment. Basically, the array functions that accept callbacks are not permitted. If in doubt just check with your instructor.

GitHub

The use of GitHub remains mandatory for this assignment. Make sure that you have already committed and tagged everything in your repository before starting this exercise.

Commits are required for each major feature created but you are free and encouraged to commit more frequently as a backup. If you have learned how to use features branches in your other courses (i.e. INFS3203 if you are taking it) you are allowed to use those features but in this course, it is not mandatory.

Database Design

In Assignment 3, the system retained a relational-style design using three collections: employees, shifts, and assignments. While this structure is appropriate in relational databases, it is not optimal for a document-oriented database such as MongoDB. Additionally we have been using employeeId and shiftId but these are not primary keys in the collection; we will remove these fields and replace them with the _id object ids from MongoDb.

We will also embed the employee ObjectId's directly into a list inside the shift objects. For example:

```
{
  "_id": {
    "$oid": "6985b8d327927b7ccfeec7b5"
  },
  "shiftId": "S001",
  "date": "2026-01-05",
  "startTime": "09:00",
  "endTime": "15:00",
  "employee": [ObjectId("6985b89f27927b7ccfeec7ad"),
ObjectId("6234b89f27937b7ccfeec7ad")]
}
```

INFS3201 – Web Technologies II
Due: March 26, 2026 @ 1159pm

Primary Key Transformation Program

In a real production system, database schema changes cannot assume that data will simply be deleted and recreated from scratch. Instead, an explicit migration process must be performed.

As part of this assignment, you must write a one-time transformation program that converts your Assignment 3 data into the new Assignment 4 schema format.

Refactor Planning Document

Before beginning implementation, you must create a planning document that describes the changes required to transition from the Assignment 3 design to the Assignment 4 design.
This document must:

- Identify which collections will change
- Identify which persistence functions must be modified or removed
- Identify any required changes to business logic functions
- Describe how embedded arrays will affect update operations
- Describe any risks or complications you anticipate

The document must be committed to GitHub before any implementation changes are made.
Name this file: A4_refactor_plan.txt and include in your project.

Suggested Process

Create a transform_db.js program with many functions. Write one function at a time in the following order. It is highly suggested that you make a backup of your database before doing these operations.

Step 1 - Create Empty Employees Array

Write a function that adds a new "employees" array into each shift. The employees array will be empty. Check in Compass to make sure that you have something like:

```
{
  "_id": {
    "$oid": "6985b8d327927b7ccfeec7b5"
  },
  "shiftId": "S001",
  "date": "2026-01-05",
  "startTime": "09:00",
  "endTime": "15:00",
  "employees": []
}
```

Step 2 - Embed Employees in Shifts

Write a function that goes through all of the assignments and adds the employee's object id (not the E001 number) to the employees array in the shift collection.

For example:

```
{
```

```
 "_id": {
  "$oid": "6985b8d327927b7ccfeec7b5"
 },
 "shiftId": "S001",
 "date": "2026-01-05",
 "startTime": "09:00",
 "endTime": "15:00",
 "employees": [
  {
   "$oid": "6985b89f27927b7ccfeec7ac"
  },
  {
   "$oid": "6985b89f27927b7ccfeec7ae"
  }
 ]
}
```


Step 3 - Remove Unnecessary Items

You can do this within compass or through the NodeJS program. If you use the shell commands instead of writing code, leave the commands in a comment in your transformation program.

1. Remove the employeeId from the employee collection.
2. Remove the shiftId from the shiftCollection.
3. Remove the assignment collection completely.

Examples:


```
{
 "_id": {
  "$oid": "6985b8d327927b7ccfeec7b5"
 },
 "date": "2026-01-05",
 "startTime": "09:00",
 "endTime": "15:00",
 "employees": [
  {
   "$oid": "6985b89f27927b7ccfeec7ac"
  },
  {
   "$oid": "6985b89f27927b7ccfeec7ae"
  }
 ]
}

{
 "_id": {
  "$oid": "6985b89f27927b7ccfeec7ac"
```

```
  },
  "name": "Alex Morgan",
  "phone": "5555-0102"
}
```

Step 4 - Make the Application Work Again

Once the data has been converted, you will need to refactor your application so that it still works with the same features as in assignment 3. You will need to find everything that used to use employeeId and shiftId and convert to _id. Remember that _id is not a string but an ObjectId so you will need statements like:

```
let empObjectId = new mongodb.ObjectId(empId)
```

Make sure to commit your work at this point.

User Authentication and Sessions

1. Create a users collection containing: username, hashed password (sha256 but salt not required). This should be done manually using Compass. You should create two accounts.
2. Leave a README.md file in your project indicating the usernames and passwords for your test users so that the person evaluating your assignment can try things.
3. Implement a login page.
    a. Invalid login attempts need to be redirected to the /login route with an appropriate message showing to the user.
    b. Valid logins should start a session and redirect the user to the / route.
4. Implement a logout page.
    a. Clear the cookie and delete the session.
5. Protect all routes except the login and logout page.
    a. If there is no session then send the user to the login page with an appropriate message.

Restrictions on implementation

- No express-sessions library permitted
- Authorization check must be done using a middleware function
- Cookie and Sessions must have a time limit of 5 minutes. Every visit by a logged in user however must extend the validity by another 5 minutes.

Security Access Log

The system must maintain a security_log collection.
Each log entry must record:

- Timestamp
- Username (if known)
- URL Accessed
- Method Used (GET, POST)

This logging operation must use a middleware function.

Submission

When submitting the zip file in D2L make sure to include the link to your GitHub repository as a comment in the submission. If you leave the link to your GitHub account or the instructor is not able to view the repository, your grade will be reduced.

Late Submissions

Late submissions will be accepted until March 28 at 11:59pm with a penalty of 50%, after which no submissions will be accepted.

INFS3201 – Web Technologies II
Due: March 26, 2026 @ 1159pm

Grading

Criteria	Exemplary	Satisfactory	Developing	Unsatisfactory
Program Features and Operation	*9 points* The program implements all features which work as expected. The user was not able to crash the program with inputs.	*7 points* The program is missing minor features or is able to cause crashes by providing specific inputs.	*5 points* The program is missing major functional features.	*0 points* The program cannot be run due to a large number of errors. The user is able to cause the program to fail with normal inputs.
Code Design and Layering	*4 points* Appropriate functions used with good parameters and return types.	*3 points* Some functions have issues in terms of parameters and return types.	*1 point* Functions were used but design was not appropriate.	*0 points* Functions were not used or parameters and return values were done using global variables.
Formatting, Documentation, and GitHub Repository	*2 points* All functions have comments that allow JSDoc to be run indicating parameter and return types. Formatting of the code is excellent. GitHub had multiple commits and a release label.	*1 point* Some functions have appropriate comments. JSDocs is not able to generate comment blocks. Formatting is mostly good but some areas could use improvement. GitHub used but not effectively.	*0 points* Only basic comments provided. Many parts of the code require reformatting.	*0 points* Code has limited or no comments. Significant formatting issues. Code was not found in GitHub.